

# 4 - Informed Search - Greedy Best-First Search

Artificial Intelligence Lab Manual - Clean Study Edition

## Learning Objectives

- Explain the role of a heuristic function.
- Implement Greedy Best-First Search with a priority queue.
- Analyze how heuristic quality affects search order and solution quality.

## Background

Informed search uses problem-specific estimates to guide exploration. Greedy Best-First Search selects the state that appears closest to the goal according to  $h(n)$ . It may be fast, but it does not generally guarantee a lowest-cost path.

## Core Concepts

| Concept           | Meaning in this lab                                                  |
|-------------------|----------------------------------------------------------------------|
| Heuristic $h(n)$  | Estimated remaining cost from node $n$ to a goal.                    |
| Priority queue    | Returns the frontier node with best priority value.                  |
| Greedy choice     | Expand the node with smallest $h(n)$ .                               |
| Heuristic quality | Affects efficiency and can mislead the search if estimates are poor. |

## Guided Procedure

1. Create a weighted or unweighted graph and assign a heuristic value to each node.
2. Use `heapq` to store frontier nodes as (heuristic, node, path).
3. Expand the lowest-heuristic node first and avoid repeated expansion.
4. Compare the expansion order with BFS.
5. Change one heuristic value to be misleading and observe the effect.

## Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import heapq

def greedy_best_first(graph, h, start, goal):
    pq = [(h[start], start, [start])]
    seen = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node in seen:
            continue
        seen.add(node)
        if node == goal:
            return path
        for nxt in graph.get(node, []):
            if nxt not in seen:
                heapq.heappush(pq, (h[nxt], nxt, path + [nxt]))
    return None
```

## Lab Exercises

**Task 1.** Construct a six-node graph and choose plausible heuristic values. Trace the algorithm manually.

**Task 2.** Create an intentionally bad heuristic that sends the search away from the goal.

**Task 3.** Compare BFS and Greedy Best-First Search by number of expanded nodes.

**Task 4.** Explain why a smaller  $h(n)$  does not necessarily imply a cheaper complete solution.

## Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

## Review Questions

1. What information is added by a heuristic?
2. Can Greedy Best-First Search return a non-optimal path? Why?
3. What makes a heuristic useful in practice?

**Completion check:** You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.